TijaraFlow

Technical Documentation

Administrator & developer guide

Version 1.0

2026 Edition · Architecture, operations, extension

Table of Contents

1. Introduction

This document targets administrators and developers who deploy, operate or extend TijaraFlow. It describes the architecture, data model, security mechanisms, integrations, deployment and operational procedures.

1.1 Scope

- Application architecture and technical choices.
- PostgreSQL data model and server functions.
- Authentication, authorization and multi-tenancy.
- Store synchronization, Stripe billing, e-mails, scheduled jobs.
- Deployment, observability and incident runbook.
- Extension procedures (new adapter, new plan, new endpoint).

1.2 Conventions

File paths are relative to the repository root. Code blocks are indicative. Environment variable names are case-sensitive.

2. Architecture overview

TijaraFlow is a Next.js 16 (App Router) application deployed on Vercel, backed by Supabase (PostgreSQL + Auth) for data and authentication, Stripe for subscriptions, and Resend for e-mails. E-commerce stores communicate via webhooks handled by an adapter layer.

2.1 Components

Component	Role
Next.js 16 (App Router)	Front-end (RSC) + back-end (Route Handlers, Server Actions).
Supabase Postgres	Database, RLS, SQL functions.
Supabase Auth (SSR)	Cookie-based authentication, server-side sessions.
Stripe	Subscriptions (Checkout + webhooks).
Resend	Transactional e-mail delivery.

Component	Role
Vercel	Hosting, continuous deployment, cron jobs.
Sync adapters	WooCommerce, Shopify (inbound webhooks).

2.2 Main flow

1. A store emits a webhook to /api/webhooks/{platform}.
2. The handler authenticates the tenant (API key), applies rate limiting, verifies the signature.
3. The adapter normalizes the payload into a common model.
4. The core (lib/sync/core) applies the mutation (product, order, customer, stock).
5. The event is logged (sync_events); eligible automations are triggered.

3. Tech stack

Area	Technology
Framework	Next.js 16, React 19, strict TypeScript
Styling	Tailwind CSS v4, shadcn/ui (Radix)
Data	PostgreSQL (Supabase), RLS enabled
Auth	Supabase Auth via @supabase/ssr
Payments	Stripe (subscription mode)
E-mail	Resend
Charts	Recharts
Hosting	Vercel (cron, env, Git deploy)

4. Repository structure

```
app/                      Routes (App Router)
  (marketing) /, /tarifs, /fonctionnalites, /contact
  login, register, reset, reset/update
  dashboard/              Authenticated area (stocks, orders, ...)
  api/v1/                 Public API (products, orders, ...)
  api/webhooks/           woocommerce, shopify, stripe
  api/billing/checkout    Create Checkout session
  api/cron/               daily, weekly
  api/export/[entity]     CSV exports
  api/health              Health probe
lib/
  supabase/               server/client + database.types
  sync/                   types, core, webhook-handler, adapters/
  entitlements.ts         plans, limits, gating
  plans.ts                plan definitions
  email/                  resend, send
  rate-limit.ts           anti-abuse
  observability.ts        error log
components/               ui (primitives), shared, app, dashboard
middleware.ts             auth guard + public routes
```

5. Environments and configuration

Environment variables are set in Vercel (Production) and, locally, in .env.local. Never commit a secret.

Variable	Role	Secret
NEXT_PUBLIC_SUPABASE_URL	Supabase project URL	No
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase public key	No
SUPABASE_SERVICE_ROLE_KEY	Service key (bypasses RLS, server only)	Yes

Variable	Role	Secret
STRIPE_SECRET_KEY	Stripe secret key	Yes
STRIPE_WEBHOOK_SECRET	Stripe webhook signing secret	Yes
STRIPE_PRICE_PRO	Pro plan price ID	No
STRIPE_PRICE_BUSINESS	Business plan price ID	No
RESEND_API_KEY	Resend API key	Yes
RESEND_FROM_EMAIL	E-mail sender (default onboarding@resend.dev)	No
CRON_SECRET	Cron authorization token	Yes
NEXT_PUBLIC_APP_URL	Public app URL	No

> **Security rule**
> SUPABASE_SERVICE_ROLE_KEY bypasses RLS. Use it only server-side (route handlers, crons, webhooks) and never expose it to the browser.

6. Data model

All business tables carry a user_id column and are protected by RLS: each user accesses only their own rows. System writes (sync, webhooks, crons) use the service key.

6.1 Main tables

Table	Content
products	Catalog: name, sku, price, cost, stock_quantity, low_stock_threshold, external_id/source, woo_id.
customers	Customers: full_name, email, phone, address.

Table	Content
orders	Orders: status, total_amount, customer_id, external_id/source.
order_items	Order lines: product_id, quantity, unit_price.
invoices	Invoices: invoice_number, status, amount, due_date, paid_at.
api_keys	API keys: key_prefix, key_hash (sha256), last_used_at.
user_settings	Settings: notify_email, auto_invoice, stock_alerts, overdue_reminders, weekly_report, wc_webhook_secret.
user_subscriptions	Subscription: plan, status, stripe_customer_id, stripe_subscription_id, current_period_end.
stock_movements	Stock ledger: delta, reason, balance_after, reference.
sync_events	Sync log: source, action, status, detail.
error_logs	Error log: source, context, message, details.
rate_limits	Anti-abuse counters: bucket, count, window_start.
demo_requests	Demo requests (landing form).
contact_messages	Contact messages (Contact page).

RLS

User-facing tables have “select/insert own” policies based on auth.uid() = user_id. sync_events and stock_movements are read-only for users; inserts go through the service key or SECURITY DEFINER functions.

7. PostgreSQL functions

Function	Signature and role
decrement_stock	(p_product_id, p_quantity, p_reason='sale', p_reference): decrements stock and logs a movement.
increment_stock	(p_product_id, p_quantity, p_reason='restock', p_reference): increments and logs.
generate_invoice_number	() → text: sequential invoice number.
check_rate_limit	(p_bucket, p_limit, p_window_seconds) → boolean: atomic fixed window, fail-open.
forecast_stock	(p_user_id, p_history_days=30, p_lead_time_days=7, p_cover_days=30) → table: velocity, cover, reorder.
reports_overview	(p_user_id, p_days=30) → jsonb: revenue, top products, stock value, customers.

The stock and reporting functions are SECURITY DEFINER and explicitly filter by user_id.

8. Authentication and authorization

8.1 Sessions

Authentication uses Supabase Auth via @supabase/ssr. The server client reads/writes session cookies; the middleware refreshes the session and protects routes.

8.2 Middleware and public routes

middleware.ts redirects any unauthenticated user to /login, except for public routes: /, /tarifs, /fonctionnalites, /contact, /login, /register, /reset, /auth, /payment, /api/webhooks, /api/v1, /api/health, /api/demo-request, /api/contact.

8.3 Password reset

6. /reset calls resetPasswordForEmail with redirectTo = /auth/callback?next=/reset/update.

7. /auth/callback exchanges the code and redirects to the internal target.
8. /reset/update calls updateUser({ password }).

8.4 Multi-tenancy

Isolation relies on user_id + RLS. System operations use the service key and filter by user_id manually.

9. Plans and entitlements

lib/plans.ts defines three plans (Starter, Pro, Business) with their limits (products, orders, stores) and features (automations, ai, api). lib/entitlements.ts enforces these rights.

Helper	Role
getUserPlan(supabase, userId)	Returns the effective plan (active subscription else Starter).
canCreate(supabase, userId, resource, qty)	Checks the product/order limit before creation.
canUseStore(supabase, userId, source)	Checks the store limit (multi-store).
hasAutomations / usersWithAutomations	Gate automation execution (Pro+).

9.1 Enforcement points

- Product/order creation (dashboard actions + API v1).
- Public API access (Business only) in withApiAuth.
- Connecting a new store in the webhook handler.
- Automation execution (core sync, orders, crons).

10. Synchronization architecture

Synchronization follows an adapter pattern: a normalized core and one adapter per platform.

```
lib/sync/types.ts          Normalized model + PlatformAdapter interface
lib/sync/core.ts           Business logic (upsertProduct, createOrder, ...)
lib/sync/webhook-handler.ts  API key auth → rate limit → signature → dispatch
lib/sync/adapters/woocommerce.ts
lib/sync/adapters/shopify.ts
lib/sync/sync-log.ts       Event logging (sync_events)
```

10.1 Handler steps

9. Tenant authentication via the api_key parameter (sha256 hash → user_id).
10. Per-tenant rate limiting (bucket wh:<userId>).
11. HMAC signature verification if a secret is configured.
12. Parsing by the adapter → normalized action.
13. Store gating (canUseStore) for new sources.
14. Dispatch to the core, then logging (success/error).

10.2 Adding a platform

15. Create lib/sync/adapters/<platform>.ts implementing PlatformAdapter (source, verifySignature, parse).
16. Create the route app/api/webhooks/<platform>/route.ts calling handleSyncWebhook(req, adapter).
17. Make the route public in middleware.ts (already covered by /api/webhooks).

11. Store integrations

Store-side configuration consists of pointing a webhook to TijaraFlow and enabling the relevant events.

`URL: https://<app>/api/webhooks/woocommerce?api_key=sf_live_xxx`

WooCommerce: product.created/updated/deleted, order.created/updated, customer.created/updated, order.deleted. Shopify: products/orders/customers create-update-delete. An HMAC signature can be enabled via user_settings.wc_webhook_secret (tenant shared secret).

12. Stripe billing

12.1 Subscription creation

app/api/billing/checkout creates a Checkout session in subscription mode. The price ID comes from STRIPE_PRICE_PRO / STRIPE_PRICE_BUSINESS. Metadata carries user_id and plan.

12.2 Stripe webhook

app/api/webhooks/stripe verifies the signature (STRIPE_WEBHOOK_SECRET) and handles: checkout.session.completed (subscription activation or invoice payment), customer.subscription.updated/deleted (status and period update).

> **Test and live modes**
>
> Stripe objects (products, prices, webhooks) are mode-specific. To go live, recreate products/prices in Live mode, provide sk_live_/whsec_ live keys and create a Live webhook.

13. E-mails (Resend)

lib/email/resend.ts initializes the client (EMAILS_ENABLED, FROM_EMAIL); lib/email/send.ts exposes the sends (stock alert, invoice, reminder, report). Sending is gracefully skipped if RESEND_API_KEY is missing.

> **Deliverability**
>
> In test mode (onboarding@resend.dev), only e-mails to the Resend account address are delivered. For production, verify a domain and set RESEND_FROM_EMAIL.

14. Scheduled jobs (crons)

Two cron routes, protected by CRON_SECRET, are triggered by Vercel Cron:

- app/api/cron/daily: move overdue invoices to “overdue” and send reminders (+7/15/30 days).
- app/api/cron/weekly: send the weekly report (Monday).

Both respect per-plan automation gating (usersWithAutomations).

```
Authorization: Bearer ${CRON_SECRET}
```

15. Observability

- error_logs + logError(): capture errors (source, context, message, details).
- sync_events: sync event log (visible in the Integrations hub).
- /api/health: health probe (checks the database) returning 200/503.
- Vercel runtime logs: for diagnosing serverless routes.

16. Rate limiting

lib/rate-limit.ts relies on the check_rate_limit function (fixed window, fail-open). Default limits: webhook 600/min, API 120/min, per tenant. Public forms (demo, contact) are limited by IP.

17. Deployment

17.1 Pipeline

The GitHub repository is connected to Vercel: every push to main triggers a build and a production deployment. The build runs the (strict) TypeScript check.

17.2 Configuration

- Set all environment variables in Vercel (Production).
- A variable change requires a redeploy to take effect.
- Configure Vercel crons for /api/cron/daily and /api/cron/weekly.

17.3 Database migrations

Schema changes (tables, functions) are applied via SQL migrations on the Supabase project. Keep migration history in the repository.

18. Local development

18. Clone the repository and install dependencies (npm install).
19. Copy variables into .env.local (Supabase keys, Stripe test, Resend).
20. Start the development server (npm run dev).

21. Apply migrations to a development Supabase project.

> **Typing tip**
>
> Regenerate Supabase types after a schema change (lib/supabase/database.types.ts) to keep a strict typing of the data layer.

19. Security

- RLS enabled on all business tables; service key restricted to the server.
- API keys hashed (sha256); never stored in clear text.
- HMAC signature for store webhooks; Stripe signature verified.
- No card data stored (delegated to Stripe).
- Secrets only in environment variables, never in code.

20. Incident runbook

A store webhook does not arrive

Check the Synchronization log, the validity of the API key in the URL, that events are enabled on the store side, and the Vercel runtime logs on /api/webhooks/*.

Stripe: “Invalid API Key”

The STRIPE_SECRET_KEY variable is incorrect (publishable instead of secret, wrong account/mode, stray space). Fix it then redeploy.

The Stripe webhook does not update the subscription

Check that the endpoint exists in the right mode (test/live), that STRIPE_WEBHOOK_SECRET matches, and the absence of 400 “signature” in the logs.

E-mails are not sent

Check RESEND_API_KEY, the notification address, and — in production — a verified domain + RESEND_FROM_EMAIL.

The build fails

Strict TypeScript error: read the Vercel build logs, fix the typing, push again. You can compile the project to reproduce the error locally.

21. Extending the platform

21.1 Add a plan

Add an entry to PLANS (lib/plans.ts) with limits, features and priceEnv; create the Stripe product/price and the matching environment variable; add the card to the Billing module.

21.2 Add an API endpoint

Create app/api/v1/<resource>/route.ts wrapping it in withApiAuth (Bearer auth + rate limit + Business gating). Filter by userId and return normalized JSON.

21.3 Add an automation

Add a boolean column to user_settings, expose it in the Automations module (gated by hasAutomations) and wire its execution at the right place (core sync or cron).

22. Appendix — Endpoint reference

Endpoint	Method	Role
/api/v1/products	GET, POST	List / create products (Business).
/api/v1/orders	GET, POST	List / create orders (Business).
/api/v1/customers	GET	List customers (Business).
/api/v1/forecast	GET	Stock forecasts (Business).
/api/webhooks/woocommerce	POST	WooCommerce webhooks.
/api/webhooks/shopify	POST	Shopify webhooks.
/api/webhooks/stripe	POST	Stripe webhooks (subscriptions).
/api/billing/checkout	POST	Subscription Checkout session.
/api/export/[entity]	GET	CSV exports (products, orders, customers, invoices, report-*).
/api/cron/daily	GET	Daily cron (reminders).
/api/cron/weekly	GET	Weekly cron (report).
/api/health	GET	Health probe.

Detailed API documentation (authentication, schemas, examples) is provided in a dedicated document.